Programme	Details	
Title	Engineering, Security & Product	
Overview		
Audience	Engineering, Product Management, Cloud Operations, IT Security	
Training days		2
Planned training hours		12
Delivery mode	Online	
Prerequisites	To be confirmed	
Learning outcomes		

Day	Date	Session timing	Module	Hours	Focus areas	Practical application	Participant outcomes	Participant output
1	To be confirmed	To be confirmed	M1 - Best Practices for Cursor & Lovable Implementation	1.5		Write rules and prompt files on a shared sample repository, then test them with a prompt that tries to break the standards.	Set up Cursor and Lovable so every developer on a shared repository gets AI output that follows ADA coding standards.	Repository rules and prompt file set
1	To be confirmed	To be confirmed	M2 - Rapid Prototyping & UI and UX Enhancement	1.75		Build from brief to working UI in Lovable and Cursor, then iterate on stakeholder feedback.	Take a written product idea to a rendered, demonstrable frontend enhancement for a sample ADA dashboard.	Working UI prototype
1	To be confirmed	To be confirmed	M3 - Automating Code Refactoring & Unit Tests	1.5		Analyse a legacy module in Cursor, refactor it, and generate tests that validate the change.	Refactor legacy code for performance and prove nothing broke using generated unit and integration tests.	Refactored module with a passing test suite
1	To be confirmed	To be confirmed	M4 - CI and CD Pipeline Optimization & Cost Reduction	1.25		Analyse historical build logs with AI and generate scripting improvements.	Find pipeline bottlenecks and tool sprawl from build logs and propose script changes that cut compute time and cost.	Ranked bottleneck list with estimated savings
2	To be confirmed	To be confirmed	M5 - Foundations of AI-Assisted Security	1.5		Redact and scope a sample codebase and server configuration, then run an AI review with an approved tool.	Share code and configuration with AI tools for security review without exposing proprietary IP.	AI-assisted security review method and checklist
2	To be confirmed	To be confirmed	M6 - Live Vulnerability Scanning	1.75		Run a live assessment on a deliberately insecure repository, patch each finding, and re-test.	Find SQL injection, cross-site scripting and authentication bypass in a live repository and generate validated patches.	Findings report with validated patch code
2	To be confirmed	To be confirmed	M7 - Process Flow Security & Threat Modelling	1.5		Map a sample ADA cloud operations architecture and generate and refine a STRIDE model with AI.	Produce a STRIDE threat model from a cloud topology map, covering ingress points, lateral movement and mitigations.	STRIDE threat model with a mitigation for each threat
2	To be confirmed	To be confirmed	M8 - Building the SecOps AI Assistant	1.25		Design the assistant, analyse sample firewall and anomaly logs, write alert logic, and present the concept.	Design a local SecOps AI assistant that analyses firewall logs and raises alerts before incidents escalate.	Assistant design, prototype on sample logs, alert rules, presentation

Day	Module	Learning outcomes
1	M1 - Best Practices for Cursor & Lovable Implementation	Set up Cursor and Lovable so every developer on a shared repository gets AI output that follows ADA coding standards.
1	M2 - Rapid Prototyping & UI and UX Enhancement	Take a written product idea to a rendered, demonstrable frontend enhancement for a sample ADA dashboard.
1	M3 - Automating Code Refactoring & Unit Tests	Refactor legacy code for performance and prove nothing broke using generated unit and integration tests.
1	M4 - CI and CD Pipeline Optimization & Cost Reduction	Find pipeline bottlenecks and tool sprawl from build logs and propose script changes that cut compute time and cost.
2	M5 - Foundations of AI-Assisted Security	Share code and configuration with AI tools for security review without exposing proprietary IP.
2	M6 - Live Vulnerability Scanning	Find SQL injection, cross-site scripting and authentication bypass in a live repository and generate validated patches.
2	M7 - Process Flow Security & Threat Modelling	Produce a STRIDE threat model from a cloud topology map, covering ingress points, lateral movement and mitigations.
2	M8 - Building the SecOps AI Assistant	Design a local SecOps AI assistant that analyses firewall logs and raises alerts before incidents escalate.

Day	Module	Assessment / success measure	Deliverable
1	M1 - Best Practices for Cursor & Lovable Implementation	Sample changes pass the standards checklist	Repository rules and prompt file set
1	M2 - Rapid Prototyping & UI and UX Enhancement	Prototype rendered and demonstrated within the lab window	Working UI prototype
1	M3 - Automating Code Refactoring & Unit Tests	Tests pass; performance change measured on the sample	Refactored module with a passing test suite
1	M4 - CI and CD Pipeline Optimization & Cost Reduction	Bottlenecks identified on sample logs; savings estimate documented	Ranked bottleneck list with estimated savings
2	M5 - Foundations of AI-Assisted Security	No marked proprietary strings exposed during the review	AI-assisted security review method and checklist
2	M6 - Live Vulnerability Scanning	Seeded critical vulnerabilities identified and correctly patched	Findings report with validated patch code
2	M7 - Process Flow Security & Threat Modelling	Seeded threats identified, each with a mitigation	STRIDE threat model with a mitigation for each threat
2	M8 - Building the SecOps AI Assistant	Seeded anomalies detected; presentation shows usable logic, relevance and feasibility	Assistant design, prototype on sample logs, alert rules, presentation

Day	Module	Scenario	Participant activity	Expected outcome	Evidence
1	M1 - Best Practices for Cursor & Lovable Implementation	Two developers on the same repository get different code styles from the same AI tool.	Write a shared rules file and prompt library, then run the same prompt in two developer setups.	Consistent AI output regardless of who prompts it.	Side-by-side outputs both pass the standards checklist.
1	M1 - Best Practices for Cursor & Lovable Implementation	The AI suggests code that breaks the team's naming and error-handling standard.	Encode the standard as a rule, re-run the prompt, and compare with the earlier result.	The known violation is no longer generated.	Before and after diff shows the violation removed.
1	M1 - Best Practices for Cursor & Lovable Implementation	A team lead needs one AI setup rolled out across the whole team.	Draft the rollout steps: rules in repo, tool settings, review workflow, and a short onboarding guide.	A new developer can adopt the setup without help.	A peer follows the guide and gets compliant output.
1	M1 - Best Practices for Cursor & Lovable Implementation	Product and platform teams need different rules on the same codebase.	Layer team-level rules on top of the shared base and test both.	Common standards with room for team-specific needs.	Each team's sample prompt passes its own checklist.
1	M2 - Rapid Prototyping & UI and UX Enhancement	A product manager wants a new filter and summary panel on the product dashboard.	Turn a written brief into a rendered frontend in Lovable.	A clickable prototype replaces a slide discussion.	Prototype rendered and demonstrated.
1	M2 - Rapid Prototyping & UI and UX Enhancement	Mid-review, stakeholders ask for a trend chart and a different layout.	Iterate the prototype with follow-up prompts and compare versions.	Change requests handled in minutes rather than a new design cycle.	Updated prototype shown with both changes applied.
1	M2 - Rapid Prototyping & UI and UX Enhancement	Engineering must decide what to keep from the prototype.	Move the prototype into Cursor and list components, states, and open questions.	Clear line between throwaway prototype and code worth keeping.	Engineering handoff notes reviewed by the group.
1	M2 - Rapid Prototyping & UI and UX Enhancement	The prototype must match brand and design rules.	Add design tokens and layout rules to the prompt and rules file.	Prototypes look consistent without manual clean-up.	Result checked against a small design checklist.
1	M3 - Automating Code Refactoring & Unit Tests	A legacy function with nested loops slows a key report.	Analyse it in Cursor, refactor for performance, and compare timings.	Faster function with the same results.	Timing comparison and passing tests.
1	M3 - Automating Code Refactoring & Unit Tests	A module has no tests and the team fears changing it.	Generate unit tests that capture current behaviour before any change.	A safety net exists before refactoring starts.	Generated tests pass on the original code.
1	M3 - Automating Code Refactoring & Unit Tests	A refactor touches an API endpoint used by other services.	Generate integration tests for the endpoint and run them after the change.	Confidence that consumers of the endpoint are unaffected.	Integration tests pass on the refactored endpoint.
1	M3 - Automating Code Refactoring & Unit Tests	A team adds an LLM feature (for example RAG over internal documents) to a product. Include if the agentic topics in the TOC are confirmed in scope.	Review the feature for prompt injection and unsafe data exposure and propose safeguards.	Team recognises AI-specific risks in code they ship.	Findings list with proposed safeguards.
1	M4 - CI and CD Pipeline Optimization & Cost Reduction	The main pipeline takes too long and blocks releases.	Analyse build logs to find the slowest stages and propose caching or parallel steps.	A ranked plan to shorten build time.	Bottleneck list backed by log evidence.
1	M4 - CI and CD Pipeline Optimization & Cost Reduction	Several overlapping linters and scanners run in the same pipeline.	Identify overlaps and propose which tools to consolidate.	Less tool sprawl and lower compute cost.	Consolidation proposal with rationale.
1	M4 - CI and CD Pipeline Optimization & Cost Reduction	Flaky tests cause repeated reruns.	Find repeated failures in logs and suggest quarantine or fix steps.	Fewer wasted reruns.	Flaky test list drawn from the logs.
1	M4 - CI and CD Pipeline Optimization & Cost Reduction	Finance asks what the pipeline improvements are worth.	Turn findings into an estimated saving with stated assumptions.	Engineering efficiency linked to operational savings.	Estimate documented with assumptions.
2	M5 - Foundations of AI-Assisted Security	A developer wants AI to review a config file that contains secrets and internal hostnames.	Redact secrets, swap hostnames for placeholders, then run the review.	Useful review without exposing sensitive values.	No marked strings appear in the AI prompt.
2	M5 - Foundations of AI-Assisted Security	Teams are unsure which AI tool is allowed for which kind of code.	Build a simple decision table: public, internal, restricted, with allowed tools for each.	Everyone knows what can go where.	Decision table reviewed and agreed.
2	M5 - Foundations of AI-Assisted Security	A server or infrastructure configuration needs a quick misconfiguration check.	Review a sample configuration with AI and verify each finding by hand.	Faster reviews with human confirmation kept in place.	Each AI finding marked confirmed or rejected.
2	M6 - Live Vulnerability Scanning	The login and search endpoints build SQL from user input.	Confirm the SQL injection in the insecure repository and patch with parameterised queries.	Injection path closed.	Exploit attempt fails after the patch.
2	M6 - Live Vulnerability Scanning	An admin route is reachable without a proper role check.	Identify the authentication bypass and generate a fix.	Access control enforced.	Unauthorised request is now rejected.
2	M6 - Live Vulnerability Scanning	A comment and profile field render unescaped user input.	Find the cross-site scripting flaw and add output encoding.	Injected script no longer runs.	Test payload displays as text.
2	M6 - Live Vulnerability Scanning	An AI-suggested patch fixes one flaw but breaks a feature.	Run the test suite against each patch and adjust the ones that fail.	Patches are validated, not trusted blindly.	All patches pass tests before sign-off.
2	M7 - Process Flow Security & Threat Modelling	A new cloud service is exposed through an API gateway.	Map the service and use STRIDE to list spoofing, tampering and exposure threats at each ingress point.	Threats identified before the service goes live.	Threat list covers every ingress point.
2	M7 - Process Flow Security & Threat Modelling	A compromised low-privilege workload could reach the database.	Trace the lateral movement path and propose segmentation and least-privilege fixes.	Blast radius of a breach reduced.	Path diagram with a mitigation at each hop.
2	M7 - Process Flow Security & Threat Modelling	The team plans to give an AI assistant access to logs and internal data.	Threat model the AI component itself, including data leakage and prompt injection.	AI features reviewed like any other system component.	Threats and mitigations added to the model.
2	M7 - Process Flow Security & Threat Modelling	Security wants a repeatable review method for future architecture changes.	Turn the exercise into a reusable prompt and checklist.	Threat modelling becomes routine.	Template used on a second sample diagram.

Day	Module	Scenario	Participant activity	Expected outcome	Evidence
2	M8 - Building the SecOps AI Assistant	A spike of denied connections comes from a single source in the firewall log.	Have the assistant summarise and classify the activity.	Analysts see the pattern in seconds.	Assistant flags the seeded source.
2	M8 - Building the SecOps AI Assistant	Outbound data volume rises sharply at an unusual hour.	Write alert logic for the anomaly and test it on sample logs.	Early warning of possible data exfiltration.	Alert fires on the seeded event.
2	M8 - Building the SecOps AI Assistant	Analysts are overwhelmed by noisy alerts.	Add thresholds and context to reduce false positives and discuss the trade-off.	Alerts worth acting on.	Alert volume drops on the sample without missing the seeded events.
2	M8 - Building the SecOps AI Assistant	The on-call analyst needs a quick incident handoff.	Have the assistant produce a short incident summary from log evidence.	Faster, clearer handoffs.	Summary includes source, timeline and suggested next step.